

players.c

```
#include <stdio.h>
#include <string.h>
#include "types.h"
#include "functions.h"

void initializePlayers(GameState game[])
{
    int i;

    strcpy(game[0].players[0].name, "Aggressive Investor");
    game[0].players[0].strategy = AGGRESSIVE_INVESTOR;

    strcpy(game[0].players[1].name, "Conservative Banker");
    game[0].players[1].strategy = CONSERVATIVE_BANKER;

    strcpy(game[0].players[2].name, "Risk Taker");
    game[0].players[2].strategy = RISK_TAKER;

    strcpy(game[0].players[3].name, "Opportunistic Trader");
    game[0].players[3].strategy = OPPORTUNISTIC_TRADER;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        game[0].players[i].position = 0;
        game[0].players[i].cash = START_MONEY;

        game[0].players[i].inJail = 0;
        game[0].players[i].jailTurns = 0;

        game[0].players[i].bankrupt = 0;

        game[0].players[i].propertiesOwned = 0;
        game[0].players[i].railwaysOwned = 0;
        game[0].players[i].utilitiesOwned = 0;

        resetLoan(game, i);

        game[0].players[i].sufferedLoss = 0;
        game[0].players[i].antiSpecRounds = 0;
    }
}

/* Decides whether a player wants to buy the property they just
   landed on. Each strategy has its own rule, from Section 3. */
int shouldBuyProperty(GameState game[], int playerIndex)
{
    int price;
    int position;

    position = game[0].players[playerIndex].position;
    price = game[0].board[position].property.purchasePrice;

    /* match the price buyProperty() actually charges (Rule-LK 31/34) */
    if(game[0].board[position].type == PROPERTY)
    {
        price = (currentMarketValue(game, position) *
                 modifierMultiplier(game, MOD_PURCHASE_PRICE,
                                     game[0].board[position].property.group, -1)) / 100;
    }
}
```

```

switch(game[0].players[playerIndex].strategy)
{
    case AGGRESSIVE_INVESTOR:
        /* buys to complete a monopoly group even if it dips below
           the usual 1000 reserve (Section 3.1) */
        if(wouldCompleteMonopoly(game, playerIndex, position))
            return 1;
        return (game[0].players[playerIndex].cash - price >= 1000);

    case CONSERVATIVE_BANKER:
        /* avoids property purchases entirely during a recession,
           but will still buy rail/utility income (Section 3.2) */
        if(game[0].board[position].type == PROPERTY &&
            isModifierActive(game, MOD_RECESSION, -1, -1))
            return 0;
        return (game[0].players[playerIndex].cash - price >=
                game[0].players[playerIndex].cash / 2);

    case RISK_TAKER:
        return (game[0].players[playerIndex].cash >= price);

    case OPPORTUNISTIC_TRADER:
        /* never buys into a group that is currently declining */
        if(game[0].board[position].type == PROPERTY &&
            modifierMultiplier(game, MOD_GROUP_VALUE,
                               game[0].board[position].property.group, -1) < 100)
            return 0;
        /* only buys cheaper properties */
        return (price <= game[0].players[playerIndex].cash * 40 / 100);

    default:
        return 0;
}
}

/* Decides whether a player is willing to spend on a house/hotel. */
int shouldConstruct(GameState game[], int playerIndex, int cost, int isHotel)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return (cash - cost >= MIN_CASH_SAFETY);

        case CONSERVATIVE_BANKER:
            /* never builds hotels while a loan is active - houses are
               still fine (Section 3.2) */
            if(game[0].players[playerIndex].loan.active && isHotel)
                return 0;
            return (cash - cost >= cash / 2);

        case RISK_TAKER:
            return (cash >= cost);

        case OPPORTUNISTIC_TRADER:
            /* builds eagerly while a construction subsidy is active */
            if(modifierMultiplier(game, MOD_CONSTRUCTION, -1, -1) < 100)
                return (cash - cost >= cash * 30 / 100);

            /* delays building while inflation pushes costs up */

```

```

        if(game[0].economy.inflationRate > 0)
            return (cash - cost >= cash * 60 / 100);

        return (cash - cost >= cash * 40 / 100);

    default:
        return 0;
}
}

int wantsLoan(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return 1;

        case CONSERVATIVE_BANKER:
            /* only borrows if bankruptcy is close */
            return (cash < MIN_CASH_SAFETY);

        case RISK_TAKER:
            return 1;

        case OPPORTUNISTIC_TRADER:
            return (cash < RICH_CASH_LINE);

        default:
            return 0;
    }
}

int wantsToRepayLoan(GameState game[], int playerIndex)
{
    int cash;
    int loanAmount;

    cash = game[0].players[playerIndex].cash;
    loanAmount = game[0].players[playerIndex].loan.amount;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return (cash >= 2 * loanAmount);

        case CONSERVATIVE_BANKER:
            return (cash >= loanAmount);

        case RISK_TAKER:
            /* rarely repays - prefers to keep using the cash */
            return (cash >= 5 * loanAmount);

        case OPPORTUNISTIC_TRADER:
            return (cash >= (loanAmount * 3) / 2);

        default:
            return 0;
    }
}

```

```

/* Rule-LK 5: whether a player wants to borrow more against new
   collateral during a Bank visit. */
int wantsIncreaseLoan(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return 1;

        case CONSERVATIVE_BANKER:
            return (cash < COMFORTABLE_CASH);

        case RISK_TAKER:
            return 1;

        case OPPORTUNISTIC_TRADER:
            return (cash < RICH_CASH_LINE);

        default:
            return 0;
    }
}

/* Rule-LK 5: whether a player wants to extend the loan term. */
int wantsExtendLoan(GameState game[], int playerIndex)
{
    int remainingRounds;
    int cash;
    int loanAmount;

    remainingRounds = game[0].players[playerIndex].loan.remainingRounds;
    cash = game[0].players[playerIndex].cash;
    loanAmount = game[0].players[playerIndex].loan.amount;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            /* prefers to borrow more rather than extend */
            return 0;

        case CONSERVATIVE_BANKER:
            return (remainingRounds <= 5 && cash < loanAmount);

        case RISK_TAKER:
            return (remainingRounds <= 10);

        case OPPORTUNISTIC_TRADER:
            return (remainingRounds <= 5);

        default:
            return 0;
    }
}

/* Rule-LK 5: whether a player wants to refinance at the current rate
   (only makes sense when the current economy rate is lower). */
int wantsRefinance(GameState game[], int playerIndex)
{

```

```

        return (game[0].economy.loanInterestRate <
                game[0].players[playerIndex].loan.interestRate);
    }

/* Which insurance policy (if any) a player wants for a property. */
InsuranceType desiredInsurance(GameState game[], int playerIndex, int propIndex)
{
    int hasHotel;
    int purchasePrice;

    hasHotel = game[0].board[propIndex].property.hotel;
    purchasePrice = game[0].board[propIndex].property.purchasePrice;

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            if(hasHotel)
                return COMPREHENSIVE_INSURANCE;
            return BASIC_INSURANCE;

        case CONSERVATIVE_BANKER:
            return COMPREHENSIVE_INSURANCE;

        case RISK_TAKER:
            /* only insures after already suffering a loss */
            if(!game[0].players[playerIndex].sufferedLoss)
                return NO_INSURANCE;
            if(hasHotel)
                return BUSINESS_INTERRUPTION;
            return BASIC_INSURANCE;

        case OPPORTUNISTIC_TRADER:
            if(purchasePrice >= EXPENSIVE_PROP_PRICE)
                return COMPREHENSIVE_INSURANCE;
            return NO_INSURANCE;

        default:
            return NO_INSURANCE;
    }
}

/* Whether a player will pay to renovate away age depreciation. */
int shouldRenovateAgeDepreciation(GameState game[], int playerIndex, int depreciationPercent)
{
    switch(game[0].players[playerIndex].strategy)
    {
        case CONSERVATIVE_BANKER:
            return (depreciationPercent >= 10);

        case OPPORTUNISTIC_TRADER:
            return (depreciationPercent >= 15);

        default:
            /* not specified for these two strategies - reasonable default */
            return (depreciationPercent >= 20);
    }
}

int shouldMaintain(GameState game[], int playerIndex, int condition, int cost)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

```

```

    if(game[0].players[playerIndex].strategy == RISK_TAKER)
    {
        /* ignores depreciation until repair becomes unavoidable */
        return (condition < 25 && cash >= cost);
    }

    return (cash - cost >= cash * 20 / 100);
}

int willingToBid(GameState game[], int playerIndex, int propIndex, int candidateBid)
{
    int marketValue;

    if(game[0].players[playerIndex].cash < candidateBid)
        return 0;

    marketValue = currentMarketValue(game, propIndex);

    switch(game[0].players[playerIndex].strategy)
    {
        case AGGRESSIVE_INVESTOR:
            return (candidateBid <= (marketValue * 120) / 100);

        case CONSERVATIVE_BANKER:
            return (candidateBid < marketValue);

        case RISK_TAKER:
            return 1;

        case OPPORTUNISTIC_TRADER:
            return (candidateBid <= (marketValue * 80) / 100);

        default:
            return 0;
    }
}

int shouldMortgage(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case CONSERVATIVE_BANKER:
            return (cash < COMFORTABLE_CASH);

        case RISK_TAKER:
            return (cash < MIN_CASH_FLOOR);

        case AGGRESSIVE_INVESTOR:
            return (cash < MIN_CASH_SAFETY);

        case OPPORTUNISTIC_TRADER:
            return (cash < TRADER_CASH_FLOOR);

        default:
            return 0;
    }
}

```

```

int shouldRedeemMortgage(GameState game[], int playerIndex, int redeemCost)
{
    return (game[0].players[playerIndex].cash - redeemCost >= redeemCost);
}

/* Whether a player pays bail after failing to roll doubles.
   The assignment doesn't specify which strategy prefers which
   option, so this is a reasonable assumption per strategy. */
int shouldPayBail(GameState game[], int playerIndex)
{
    int cash;

    cash = game[0].players[playerIndex].cash;

    switch(game[0].players[playerIndex].strategy)
    {
        case CONSERVATIVE_BANKER:
            return (cash >= BAIL_CASH_SPARE);

        case RISK_TAKER:
            return 0;

        case AGGRESSIVE_INVESTOR:
        case OPPORTUNISTIC_TRADER:
            return (cash >= RICH_CASH_LINE);

        default:
            return 0;
    }
}

```